IFT 211 — Digital Logic Design

Comprehensive Course Notes

University of Port Harcourt

Topics: Data Representation | Number Systems | Logic Gates | Boolean Algebra | Binary Arithmetic | Computer Architecture

1. Electricity and Binary Logic

The discovery of electricity fundamentally transformed computation by providing a mechanism to represent two distinct physical states. These two states — high voltage and low voltage — map directly onto the binary digits 1 and 0.

- High voltage level represents binary 1
- Low voltage level represents binary 0
- These binary states form the backbone of all digital electronics
- Using only two voltage levels makes circuits robust, reliable, and less susceptible to electrical noise

Binary logic enables digital systems to encode, store, and process information efficiently. Any data — numbers, text, images, audio — can ultimately be expressed as a sequence of 0s and 1s.

> 📝 *Note: This is why computers work with bits (binary digits). All processing in a CPU fundamentally reduces to manipulating sequences of 0s and 1s.*

2. Data Representation

Data representation refers to the method by which a computer organizes and processes natural numbers and other data types. Computers use various number systems internally, and understanding these is fundamental to digital design.

2.1 Number Systems Overview

The four primary number systems used in digital systems are:

System	Base	Digits Used	Prefix / Notation
Decimal	10	0–9	None (everyday use)
Binary	2	0, 1	0b or subscript 2
Octal	8	0–7	0o or subscript 8
Hexadecimal	16	0–9, A–F	0x or subscript 16

2.2 Instruction Set Architecture (ISA)

The ISA refers to the interface between a computer's hardware and software. It defines what instructions a processor can execute and how data is structured.

- The ISA determines data grouping (how bits are organized into usable units)
- Word length sizes: 8-bit, 16-bit, 32-bit, 64-bit, etc.

- The instruction set that enters the system must obey the word length to receive, store and work on data correctly
- Knowing the grouping of the system helps in networking (interfacing)
- Data capture depends on the ISA
- Grouping types include: Integer, Real, and other data types

The ISA-level design of a system determines the grouping; other factors help in understanding it.

2.3 Key ISA Terminology

Op-code (Operation Code): The part of an instruction that specifies the operation to be performed (e.g., ADD, SUB, LOAD).

Operands: The data values or memory addresses that the instruction operates on.

Operation Code / Op-code: Must have instructions — showing the number of bits going in.

For holding values: There must be a declaration of the space (in which a value is occupied based on the bits).

2.4 Signed Number Representation

When referring to a signed binary representation, you take the Signed Magnitude Representation, where the Most Significant Bit (MSB) indicates the sign.

- MSB = 0 means the number is positive (+)
- MSB = 1 means the number is negative (-)

Example (4-bit representation):

```
0111  =  +7  (positive 7 in decimal)
1111  =  -7  (negative 7 in decimal, considering sign-magnitude)
```

> 📝 *Note: Signed numbers have impact on the word length. Always identify the MSB before interpreting a signed binary value.*

3. Number System Conversions

Converting between different number bases is a core skill in digital logic. The main conversion directions are:

- Decimal ↔ Binary
- Decimal ↔ Octal
- Decimal ↔ Hexadecimal
- Binary ↔ Octal ↔ Hexadecimal (direct grouping methods)

3.1 Decimal to Binary Conversion

Method: Repeatedly divide the decimal number by 2 and record the remainders. Read remainders from bottom to top (most significant to least significant bit).

Example: Convert 274 to Binary

```
274 ÷ 2 = 137  r 0   (LSB)
137 ÷ 2 =  68  r 1
 68 ÷ 2 =  34  r 0
 34 ÷ 2 =  17  r 0
 17 ÷ 2 =   8  r 1
  8 ÷ 2 =   4  r 0
  4 ÷ 2 =   2  r 0
  2 ÷ 2 =   1  r 0
  1 ÷ 2 =   0  r 1   (MSB)
```

Reading remainders from bottom (MSB) to top (LSB):

$274_{10} = 100010010_2$

> 📝 *Note: The leftmost bit (MSB = most significant bit) is the last remainder; the rightmost bit (LSB = least significant bit) is the first remainder.*

3.2 Binary to Decimal Conversion

Method: Multiply each bit by 2 raised to its positional power, then sum all the results.

Example: Convert 100010010_2 to Decimal

$1\times2^8 + 0\times2^7 + 0\times2^6 + 0\times2^5 + 1\times2^4 + 0\times2^3 + 0\times2^2 + 1\times2^1 + 0\times2^0$

$= 256 + 0 + 0 + 0 + 16 + 0 + 0 + 2 + 0 = 274_{10}$ ✓

3.3 Decimal to Octal Conversion

Method: Repeatedly divide by 8 and record remainders. Read from bottom to top.

Example: Convert 274 to Octal

```
274 ÷ 8 = 34  r 2
 34 ÷ 8 =  4  r 2
  4 ÷ 8 =  0  r 4
```

$274_{10} = 422_8$

Verification:

$4\times8^2 + 2\times8^1 + 2\times8^0 = 4\times64 + 2\times8 + 2\times1 = 256 + 16 + 2 = 274$ ✓

3.4 Decimal to Hexadecimal Conversion

Method: Repeatedly divide by 16 and record remainders. Read from bottom to top.

Example: Convert 274 to Hexadecimal

```
274 ÷ 16 = 17  r 2
 17 ÷ 16 =  1  r 1
  1 ÷ 16 =  0  r 1
```

$274_{10} = 112_{16}$

Verification (checking 0011 in base 16):

$3 \times 16^{-1} = 3 \times (1/16) = 3/16 = 0.1875$

3.5 Converting Fractional Numbers

For the fractional part, repeatedly multiply by the target base and record the integer part of the result (read from top to bottom).

Example: Convert 274.1875_{10} to Binary

Integer part: $274_{10} = 100010010_2$ (as shown above)

Fractional part (0.1875):

```
0.1875 × 2 = 0.3750  → 0
0.3750 × 2 = 0.7500  → 0
0.7500 × 2 = 1.5000  → 1
0.5000 × 2 = 1.0000  → 1
```

$0.1875_{10} = .0011_2$

Therefore: $274.1875_{10} = 100010010.0011_2 = 422.14_8 = 112.3_{16}$

3.6 Binary to Octal (Grouping Method)

Group binary digits into sets of 3 from the binary point outward (add leading zeros if needed). Each group of 3 bits represents one octal digit.

Since $2^3 = 8$:

```
100 010 010 . 001 100
 4    2   2  .  1   4
= 422.14₈  ✓
```

Checking: $001 = 1\times2^0 = 1$, $100 = 1\times2^2 = 4$

$\therefore .14_8 = (1\times8^{-1}) + (4\times8^{-2}) = 1/8 + 4/64 = 8/64 + 4/64 = 12/64$
$= 0.1875$ ✓

3.7 Binary to Hexadecimal (Grouping Method)

Group binary digits into sets of 4 from the binary point outward. Each group of 4 bits represents one hex digit.

Since $2^4 = 16$:

```
0001  0001  0010 . 0011
  1     1     2  .  3
= 112.3₁₆  ✓
```

Verification: $0011_2 = 1\times2^{-3} + 1\times2^{-4} = 1/8 + 1/16 = 2/16 + 1/16 = 3/16 = 0.1875$ ✓

4. Binary Arithmetic

Binary arithmetic follows the same principles as decimal arithmetic but uses only two digits: 0 and 1.

4.1 Binary Addition

Rules for binary addition:

A	B	Sum (A+B)	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Key rule: 1 + 1 = 10 in binary (sum = 0, carry = 1), similar to how 9 + 1 = 10 in decimal.

Example: Add A = 5 and B = 4 in binary (4-bit system)

$$A = 5 = 0101_2$$
$$B = 4 = 0100_2$$
$$\text{Sum} = 1001_2 = 9_{10} \checkmark$$

Larger example: 5 + 4 in an 8-bit system:

$$0000\ 0101 + 0000\ 0100 = 0000\ 1001 = 9 \checkmark$$

4.2 Binary Subtraction

Rules for binary subtraction:

A	B	Difference (A-B)	Borrow
0	0	0	0
0	1	1	1 (borrow)
1	0	1	0
1	1	0	0

Key rule: 0 - 1 requires borrowing from the next higher bit (like borrowing 10 in decimal).

Example: 40 - 5 in binary (8-bit)

```
40₁₀ = 00101000₂
 5₁₀ = 00000101₂
──────────────────
35₁₀ = 00100011₂
```

Verification: $00100011_2 = 32 + 2 + 1 = 35$ ✓

Another example: $101 \div 5 = 101_{10} \div 5_{10}$ in binary:

```
101₁₀  =  1100101₂
  5₁₀  =     101₂
1100101 ÷ 101 = 10100₂  (repeatedly subtract and shift)
= 20₁₀  ✓  (101 ÷ 5 = 20.2, integer part = 20)
```

📝 *Note: In binary division: if divisor ≤ remaining dividend, quotient bit = 1 and subtract; otherwise quotient bit = 0 and bring down next bit.*

4.3 Binary Multiplication

Binary multiplication uses the same long multiplication method as decimal, but simpler since each multiplier bit is either 0 or 1.

Notation: For two n-bit numbers A and B:

$$A = A_{n-1}\ A_{n-2}\ \ldots\ A_1\ A_0$$
$$B = B_{n-1}\ B_{n-2}\ \ldots\ B_1\ B_0$$

Partial products are formed by multiplying A by each bit of B, shifted appropriately.

Example: 21 × 17 in binary

```
21₁₀  =  10101₂
17₁₀  =  10001₂

Partial Products:
  10101   × 1  (bit 0 of 17)  →  00010101
  10101   × 0  (bit 1 of 17)  →  00000000  (shifted 1)
  10101   × 0  (bit 2 of 17)  →  00000000  (shifted 2)
  10101   × 0  (bit 3 of 17)  →  00000000  (shifted 3)
  10101   × 1  (bit 4 of 17)  →  10101     (shifted 4)  →  101010000

Sum of partial products:
  000010101
+ 101010000
────────────
  101100101₂
```

$101100101_2 = 1\times2^8 + 0\times2^7 + 1\times2^6 + 1\times2^5 + 0\times2^4 + 0\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0$

$= 256 + 64 + 32 + 4 + 1 = 357_{10}$

$21 \times 17 = 357$ ✓

📝 *Note: The sum of all partial products gives the final product. This is how hardware multipliers work.*

4.4 Binary Division

Binary division is analogous to long division in decimal. At each step, determine if the divisor fits into the current portion of the dividend.

Example: $101_{10} \div 5_{10}$ in binary

$101_{10} = 1100101_2$

$5_{10} = 101_2$

Step-by-step long division:

```
1100101 ÷ 101:
  110 ÷ 101  →  1 (quotient bit), remainder 001
  0010 ÷ 101 →  0 (quotient bit), remainder 0010
  00101 ÷ 101 → 1 (quotient bit), remainder 000
  Quotient  =  10100₂  =  20₁₀  ✓
```

5. Logic Gates

Logic gates are the fundamental building blocks of digital circuits. They perform Boolean operations on one or more binary inputs to produce a single binary output. Every complex digital system — from a simple calculator to a modern CPU — is built from combinations of these basic gates.

5.1 AND Gate

The AND gate performs logical multiplication. Its output is 1 only when ALL inputs are 1.

Boolean Expression: $C = A \cdot B$ (also written A AND B or $A \times B$)
Symbol: A flat-fronted gate with a curved output side (D-shape)

AND Gate Truth Table:

A	B	C = A·B
0	0	0
0	1	0
1	0	0
1	1	1

📝 *Note: AND is like multiplication: 1×1=1, but 1×0=0. Output is 1 only if BOTH inputs are 1.*

5.2 OR Gate

The OR gate performs logical addition. Its output is 1 when ANY input is 1.

Boolean Expression: $C = A + B$ (also written A OR B)
Note: $A + B = C$, so $C = A + B$

OR Gate Truth Table:

A	B	C = A+B
0	0	0
0	1	1
1	0	1
1	1	1

Key difference from AND: OR outputs 1 if at least one input is 1. The only time OR outputs 0 is when ALL inputs are 0.

5.3 NOT Gate (Inverter)

The NOT gate negates (inverts) its single input. It has only one input and produces the complement.

Boolean Expression: C = A' or C = Ā (A-bar) — read as 'NOT A'
Symbol: A triangle with a small circle (bubble) at the output

NOT Gate Truth Table:

A	C = A'
0	1
1	0

> 📝 *Note: The NOT gate is the simplest gate — it simply flips the bit. It is the only gate with a single input.*

5.4 NAND Gate (NOT-AND)

The NAND gate is the complement of the AND gate. Its output is 0 only when ALL inputs are 1 (otherwise output is 1).

Boolean Expression: C = (A·B)' = A·B with a bar over it
Symbol: AND gate with a bubble at the output

NAND Gate Truth Table:

A	B	C = (A·B)'
0	0	1
0	1	1
1	0	1
1	1	0

NAND is a universal gate — any other logic function can be built using only NAND gates. This is very important for circuit implementation.

5.5 NOR Gate (NOT-OR)

The NOR gate is the complement of the OR gate. Its output is 1 only when ALL inputs are 0.

Boolean Expression: C = (A+B)' = A+B with a bar over it
Symbol: OR gate with a bubble at the output

NOR Gate Truth Table:

A	B	C = (A+B)'
0	0	1
0	1	0
1	0	0
1	1	0

Like NAND, NOR is also a universal gate.

5.6 XOR Gate (Exclusive OR)

The XOR gate outputs 1 when inputs are DIFFERENT. When both inputs are the same, it outputs 0. XOR is sometimes called 'Exclusive OR.'

Boolean Expression: $C = A \oplus B$ (A XOR B)
XOR signal is often described as 'getting 1 when there is only an odd number of 1s in the input.'

XOR Gate Truth Table:

A	B	C = A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

📝 *Note: XOR is 0+0=0, 0+1=1, 1+0=1, 1+1=0. It outputs 1 only when inputs DIFFER.*

5.7 XNOR Gate (Exclusive NOR)

The XNOR gate is the complement of XOR. It outputs 1 when inputs are the SAME.

Boolean Expression: $C = A \odot B = (A \oplus B)'$

XNOR Gate Truth Table:

A	B	C = A⊙B
0	0	1
0	1	0
1	0	0
1	1	1

Exclusive NOR (XNOR) is sometimes called 'coincidence gate' — it outputs 1 when inputs coincide (are equal).

5.8 Combined Logic Expressions

Logic gates can be combined to implement complex Boolean expressions. When multiple gates are connected:

- Each gate's output can feed as input to another gate
- The overall output depends on the combined Boolean expression

Example: (A+B) + (C+D) then A+B

This shows three OR gates: first OR(A,B), second OR(C,D), then final OR combining both outputs with another A+B signal.

6. Boolean Algebra and De Morgan's Theorems

Boolean algebra formalizes rules for manipulating binary variables. Using AND, OR, and NOT, engineers can simplify circuits, reducing gate count and power consumption.

6.1 Boolean Algebra Basics

Computer (digital) system architecture is the knowledge/idea for applying all the logical components. These components are designed to work with digital signals and perform modulation and conversion, applying logic units (ALUs).

Variables: Boolean variables can only take values 0 or 1
Operators: x = · (AND operator), + = OR operator
NOT operator: A bar (overline) or apostrophe denotes complement

Boolean Postulates:

- Addition: 0+0=0, 0+1=1, 1+0=1, 1+1=1
- Multiplication: 0·0=0, 0·1=0, 1·0=0, 1·1=1
- Complement: If A=0 then A'=1; if A=1 then A'=0

6.2 Truth Table

A truth table shows all possible input combinations and the resulting output for a Boolean expression.

- The truth table has individual rows that show the fixed amount of input variables
- If there are n variables, there are 2^n possible input combinations
- Examples: 2 variables → 2^2 = 4 rows; 3 variables → 2^3 = 8 rows

Example: Addition (F = A + B → OR-operator) with B → output is C:

A	B	C = A+B
0	0	0
0	1	1
1	0	1
1	1	1

6.3 De Morgan's Theorems

De Morgan's Theorems provide rules for simplifying expressions involving NOT applied to AND or OR combinations. They are essential tools for circuit simplification.

Theorem 1 (NOT-AND → OR of NOTs):

```
NOT(A · B)  =  NOT A  +  NOT B
```

```
(A·B)'  =  A' + B'
```

Theorem 2 (NOT-OR → AND of NOTs):

```
NOT(A + B)  =  NOT A  ·  NOT B
(A+B)'  =  A' · B'
```

Example Application: Simplify NOT(A + B·C)

```
NOT(A + B·C)  →  NOT A · NOT(B·C)
              →  NOT A · (NOT B + NOT C)
              =  A' · (B' + C')
```

📝 *Note: De Morgan's theorems tell us: to complement a function, swap AND↔OR and complement each variable.*

7. Flip-Flops

Flip-flops are memory elements that store 1 bit of information. They are the fundamental building blocks of sequential circuits — circuits whose output depends not just on current inputs but also on past states (memory).

Unlike combinational circuits (logic gates), flip-flops are synchronized by a clock signal and change state only on clock edges.

7.1 Types of Flip-Flops

SR Flip-Flop (Set-Reset)

- Two inputs: S (Set) and R (Reset)
- S=1 sets output Q to 1; R=1 resets Q to 0
- S=0, R=0: Q retains its previous state (memory)
- S=1, R=1: Forbidden state (undefined/invalid)

D Flip-Flop (Data/Delay)

- Single data input D
- On the active clock edge, Q takes the value of D
- Eliminates the forbidden state of SR flip-flop
- Most commonly used flip-flop in practice
- Used in shift registers and data storage

How a D flip-flop stores a bit: On each rising (or falling) clock edge, the flip-flop 'samples' the D input and stores it in Q. The stored value remains until the next clock edge.

JK Flip-Flop

- Inputs: J (analogous to Set) and K (analogous to Reset)
- J=1, K=0: Q is set to 1
- J=0, K=1: Q is reset to 0
- J=0, K=0: Q retains previous state
- J=1, K=1: Q toggles (flips) — key advantage over SR flip-flop
- No forbidden state — most versatile flip-flop

T Flip-Flop (Toggle)

- Single input T (Toggle)
- T=0: Q retains previous state
- T=1: Q toggles (complement of previous state)
- Can be built from a JK flip-flop with J=K=T
- Used in binary counters

7.2 Applications of Flip-Flops

- Registers: Groups of flip-flops that store multi-bit values
- Counters: Sequential flip-flops that count clock pulses
- Shift registers: Data shifted in/out serially
- Memory elements in sequential logic
- State machines: Core storage in finite state machines

8. Computer System Components

A computer system consists of hardware and software working together. Understanding these components is essential for digital systems design.

8.1 Hardware Components

CPU (Central Processing Unit): The 'brain' of the computer. Performs all computations and controls other components.

Memory: Stores instructions and data. Includes RAM (volatile) and ROM (non-volatile).

I/O Devices: Input/Output peripherals: keyboard, mouse, display, storage drives, etc.

8.2 Software

Operating System (OS): Manages hardware resources, provides user interface, and runs applications.

Applications: Programs that perform specific user tasks (word processors, browsers, etc.).

Firmware: Low-level software embedded in hardware that controls it directly (e.g., BIOS).

8.3 CPU Internal Components

Component	Full Name	Function
CU	Control Unit	Directs and coordinates all CPU operations; fetches and decodes instructions
ALU	Arithmetic Logic Unit	Performs all arithmetic (+, -, ×, ÷) and logical (AND, OR, NOT) operations
Registers	CPU Registers	Small, ultra-fast storage locations inside the CPU for temporary data
Cache	CPU Cache	Small, fast memory close to CPU to reduce memory access time

8.4 Buses

Buses are communication pathways that transfer data between components:

- Data Bus: Carries actual data between CPU, memory, and I/O
- Address Bus: Carries memory addresses (where to read/write data)
- Control Bus: Carries control signals (read/write, interrupts, clock)

> 📝 *Note: The width of the data bus determines how many bits can be transferred at once — 32-bit or 64-bit buses are common in modern systems.*

9. Instruction Cycle (Fetch-Decode-Execute)

The instruction cycle describes the sequence of steps a CPU follows to process each instruction. Every instruction, from a simple addition to a complex division, goes through this cycle.

9.1 The Three Stages

Stage 1: Fetch

- The CPU reads the next instruction from memory
- The Program Counter (PC) holds the address of the instruction
- Instruction is loaded into the Instruction Register (IR)
- PC is incremented to point to the next instruction

Stage 2: Decode

- The Control Unit (CU) interprets the instruction in the IR
- Identifies the opcode (what operation to perform)
- Identifies the operands (what data to use)
- Prepares the appropriate functional units

Stage 3: Execute

- The ALU or appropriate unit carries out the operation
- Results are written back to registers or memory
- For I/O instructions, data is transferred to/from peripherals

9.2 Example

Instruction: ADD R1, R2 → R3

- Fetch: CPU fetches the ADD instruction from memory
- Decode: CU identifies it as an ADD operation with operands R1 and R2, result to R3
- Execute: ALU adds the values in R1 and R2; result stored in R3

10. Assignment / Practice Problems

These problems are drawn from the course material and are meant for exam practice.

10.1 Assignment Problem

Using a matriculation number ending with 3 digits, let those digits represent values A, B, C (representing numbers from 0–9). Convert EAC.BCA to:

- Base 2 (Binary)
- Base 8 (Octal)
- Base 16 (Hexadecimal)

Example approach (with A=E, B=C, C=A from hex — representing decimal digits):

- First, interpret EAC.BCA as a hexadecimal or custom representation of digits
- Convert each digit to its decimal equivalent
- Then convert the full number to the required bases

10.2 Practice Questions

1. Convert 147_{10} to binary, octal, and hexadecimal.
2. Perform binary addition: 10110101 + 01101011.
3. Draw the truth table for the expression: F = A·B + A'·C.
4. Apply De Morgan's theorem to simplify: NOT(A·B + C).
5. Explain how a D flip-flop stores a bit and draw its timing diagram.
6. Multiply $13_{10} \times 11_{10}$ using binary multiplication.
7. Describe the three stages of the instruction cycle with an example.

Quick Reference Summary

Number Base Conversions

Conversion	Method	Grouping Size
Binary → Octal	Group 3 bits from binary point outward	3 bits = 1 octal digit
Binary → Hex	Group 4 bits from binary point outward	4 bits = 1 hex digit
Decimal → Binary	Divide by 2, collect remainders bottom-up	—
Decimal → Octal	Divide by 8, collect remainders bottom-up	—
Decimal → Hex	Divide by 16, collect remainders bottom-up	—
Fraction → Binary	Multiply by 2, collect integer parts top-down	—

Logic Gate Summary

Gate	Symbol	Expression	Output = 1 when...
AND	D-shape	A · B	ALL inputs are 1
OR	Curved shape	A + B	ANY input is 1
NOT	Triangle + bubble	A'	Input is 0
NAND	AND + bubble	(A·B)'	NOT all inputs are 1
NOR	OR + bubble	(A+B)'	ALL inputs are 0
XOR	OR with curves	A ⊕ B	Inputs are DIFFERENT
XNOR	XOR + bubble	(A⊕B)'	Inputs are SAME

Key Formulas

- Number of rows in a truth table with n variables = 2^n
- Binary to Decimal: Sum of (bit × 2^position)
- Fractional binary to decimal: Sum of (bit × 2^negative_position)
- De Morgan 1: (A·B)' = A' + B'
- De Morgan 2: (A+B)' = A' · B'
- MSB = 0 → positive number; MSB = 1 → negative (signed magnitude)

— End of IFT 211 Digital Logic Design Notes —